Problem Formulation: Intelligent Traffic Signal Coordination Using Multi-Agent Reinforcement Learning

Technical brief: MDP formulation, algorithm and multi-agent interaction | custom Python simulator (not SUMO)

1. Problem statement and motivation

Urban congestion wastes time and fuel. Conventional fixed-time signals repeat a pre-set cycle regardless of the actual traffic, so green time is wasted on empty approaches while long queues build elsewhere, and queues at one junction spill back and block the next. The task is to let **every signal learn, from experience, which phase to display** so that vehicles wait less across the whole network, while neighbouring signals exchange information instead of acting blindly. This is a sequential decision problem under uncertain demand, hence a natural fit for RL.

2. Environment

Custom Python simulator (*not* SUMO): a 3x3 grid of 9 two-phase signalised intersections joined by links of 8 s travel time, 12 boundary entry points and individually tracked vehicles on fixed routes. A green approach discharges one vehicle per 2 s; switching phase costs 2 s of all-red; an approach that is full (20 vehicles) blocks the upstream junction (spill-back). Episode = 1800 s; an agent acts every 10 s (180 decisions). Five demand scenarios (low, medium, high, asymmetric rush-hour, time-varying peak).

Component	Definition for agent i (one per intersection)
State s_i	(queue of N+S axis, queue of E+W axis) each in 4 levels {0-3, 4-9, 10-19, 20+} x current phase (2) x longest head-of-queue wait of the red axis in 3 levels {<15, 15-40, >40 s} = 96 local states; **Coop-IQL** adds 2 binary flags 'downstream approach of axis is ≥60% full' (received from neighbours) = 384 states.
Action a_i	{0 = north-south green, 1 = east-west green} held for the next 10 s (a change inserts 2 s all-red).
Reward r_i	$r_i = -(0.10 \cdot q_{avg,j} + 0.50 \cdot w_{max,i}/60 + 0.50 \cdot \min(b_i/5, 2))$, with q_{avg} = mean number of queued vehicles at the junction over the interval, w_{max} = longest head-of-queue waiting time (s), b = blocked discharges (spill-back). Penalises queues, waiting and congestion.
Transition	Stochastic: arrivals are random and the next state also depends on neighbours' phases; discount $\gamma = 0.9$.

3. RL algorithm

Tabular **Independent Q-Learning (IQL)**: each of the 9 agents keeps its own table $Q_i(s,a)$ and updates $Q_i(s,a) \leftarrow Q_i(s,a) + \alpha[r + \gamma \max_{a'} Q_i(s',a') - Q_i(s,a)]$, $\alpha = 0.1$, ε-greedy exploration decayed linearly from 1.0 to 0.05 over the first 70% of 1000 training episodes (random demand scenario each episode). Tabular learning keeps the system small, interpretable and reproducible on a laptop.

4. Multi-agent interaction

The 9 agents act *simultaneously*; the environment is therefore non-stationary for each agent (the neighbours are learning too), which is the central difficulty of IQL. **Cooperation** is added in two ways without a central controller (centralised training would not scale): (a) *information sharing* - each agent observes how full the approaches it feeds into are; (b) *reward sharing* - $\tilde{r}_i = 0.7\, r_i + 0.3 \cdot \text{mean}_{j \in N(i)} r_j$, so a junction is penalised for pushing congestion onto its neighbours. Plain IQL (no sharing), Fixed-time (20 s green), Random and Longest-queue-first are compared on identical demand.

5. Objective of optimisation

Each agent maximises $E[\Sigma_t \gamma^t r_{i,t}]$; jointly this approximates the network goal of **minimising average vehicle waiting time and queue length while maximising throughput**: $\min_\pi E[(1/|V|) \Sigma_v W_v]$ with W_v = total time vehicle v spends queued. Evaluation metrics: average waiting time, average queue length, throughput, average travel time, congestion index.